

CryptoBench-SC：面向对称密码分析智能体研究能力的认知动作驱动基准

说明：本文为中文版论文草稿。引言部分基本沿用你提供的写作逻辑；方法、构建、评测协议与实验部分已按科研论文写法重写。所有出现“（占位假设，待真实实验替换：.....）”的数值、规模、实验结果和结论均为写作占位，正式投稿前必须由真实数据替换。

摘要

随着大语言模型与工具型智能体的发展，智能体开始被用于辅助密码学论文理解、攻击代码生成、求解器调用与实验结果整理。然而，现有密码学评测多关注孤立知识、经典解密题或 CTF 式求解，难以评估智能体是否具备完成对称密码分析研究工作流的能力。本文提出 **CryptoBench-SC**，一个面向对称密码分析智能体的细粒度基准。不同于按差分、线性、积分或代数攻击等分析对象组织题目，CryptoBench-SC 以人类密码分析研究员的核心认知动作作为设计基础，将研究能力分解为理解、判断、构造、验证、诊断与论证六类动作，并进一步实例化为七种题型：Primitive 理解与结构化、攻击描述还原、分析方案规划、攻击实现与验证、声明独立验证、失败诊断与修复、研究报告撰写。每道题均被组织为包含输入资源、产物结构、隐藏验证标签、验证器映射和评分聚合规则的 task package，并通过确定性验证器、可执行 oracle、证据绑定检查和保真度约束进行自动评分。（占位假设，待真实实验替换：CryptoBench-SC v1.0 包含 180 道任务，覆盖 6 类认知动作、7 种题型、5 类主要密码分析内容标签与 3 个难度层级。）我们对多个主流智能体系统进行评测，发现当前智能体在规范理解与结构化方面已具备一定能力，但在失败修复、独立验证和证据约束下的结论表达方面仍显著落后于人类研究员。（占位假设，待真实实验替换：最强智能体 overall score 为 41.8/100，人类专家基线为 82.7/100。）这些结果表明，对称密码分析智能体应被作为研究工作流执行者进行评估，而非作为单轮问答模型进行评估。

1. 引言

对称密码分析长期以来被视为高度依赖人类专家的研究工作。研究人员需要阅读最新的密码方案与攻击论文，理解其设计原理与威胁模型，选择并配置合适的分析工具，设计实验，修复失败，并最终撰写有证据支撑的研究结论。这一系列任务不仅要求扎实的理论功底，更依赖大量的工程经验与判断力，因此长期被视为难以自动化的研究领域之一。然而，随着大语言模型与智能体技术的快速发展，这一格局正在发生变化。近期已有若干研究尝试以智能体形式辅助甚至独立完成对称密码分析任务，这类系统能够阅读论文、生成攻击代码、调用密码学工具链，并对实验结果进行初步分析。这些工作表明，智能体在对称密码分析这一专业领域中已从概念验证阶段迈向实际应用的早期探索。

上述进展引出了一个无法回避的根本问题：在对称密码分析这一专业领域中，智能体距离真正的研究员还有多远？回答这一问题对于该领域的健康发展至关重要。在当前实践中，研究团队在多个智能体框架之间进行选型时，缺乏客观的评估依据；新发布的智能体框架对外宣称具备密码分析能力时，社区缺乏统一的标准用以核验该主张；不同研究者在讨论何为更强的智能体时，所指标准亦不一致。在缺乏公认评测体系的情况下，该领域的发展将不得不依赖主观判断与经验直觉。学术界已意识到此类空白，并陆续提出若干面向密码学的评测工作，例如 AICrypto 与 CipherBank 分别从密码学知识掌握度和密码题目求解能力等角度对大语言模型进行了评估，为社区提供了重要的早期工具。

然而，上述工作不足以回答“智能体是否具备研究能力”这一问题，其根本原因在于一次尚未被充分识别的范式转变。现有 benchmark 的评测对象通常是大语言模型，而大语言模型本质上更接近单次问答系统：输入一个问题，输出一个答案。智能体则与之不同。智能体是一类能够调用工具、进行多步推理、维护中间状态，并在失败后修正行为的研究流程执行者。以单次问答为前提设计的评测体系，无法刻画智能体在工具使用、流程规划、错误修复、证据追溯等维度上的真实表现；以此类 benchmark 评估智能体，所测得的并非智能体的整体研究能力，而仅是其在单次问答情境下最接近大语言模型的那一部分能力。

这种评测对象与评测体系的错配具体表现为三个层面的局限。第一，现有 benchmark 在对称密码分析领域内的覆盖较为零散，关键子领域的处理不够均衡，缺乏一个系统性的领域知识结构作为评测骨架。第二，现有 benchmark 测的是孤立解题能力，而研究人员的实际工作并非孤立解题，而是完成一套完整研究工作流：从论文理解到结构化表示，从需求到规划，从工具调用到执行，从失败到修复，从结果到报告。第三，现有 benchmark 大多仅以最终答案作为评分依据，这种方式难以区分一个真正理解问题的智能体与一个偶然得出正确答案的智能体。优秀设计与劣质设计的差异往往出现在研究过程中，而非仅出现在终点答案中。

针对上述局限，本文提出 CryptoBench-SC，一个面向对称密码分析智能体研究工作流的细粒度评测框架。

CryptoBench-SC 的核心思想不是增加更多密码学题目，而是重新定义评测对象：我们将智能体视为研究工作流执行者，并将其能力拆解为一组可观察、可提交、可验证的研究产物。具体而言，我们从人类密码分析研究员的核心认知动作出发，将任务组织为七类题型，并为每类题型定义输入形式、输出工件、隐藏标签、验证器和评分规则。由此，CryptoBench-SC 将“智能体是否能完成密码分析研究”这一抽象问题转化为一组具体问题：它是否能读懂规范和论文，是否能正确形式化分析目标，是否能选择合适策略，是否能生成可执行工件，是否能独立验证结果，是否能修复失败，是否能写出证据边界清晰的研究报告。

本文的主要贡献如下。第一，我们提出一种面向智能体研究工作流的评测方法论，将智能体能力评测从“是否得出正确答案”重塑为“是否产生支撑研究结论所需的中间证据链”。第二，我们构建 CryptoBench-SC，一个面向对称密码分析智能体的细粒度评测框架，包含任务包结构、隐藏标签、自动验证器、可执行 oracle、证据绑定检查和人类基线。第三，我们通过对多个智能体系统的评测，量化当前智能体在对称密码分析研究工作流中的能力边界与失败模式。（占位假设，待真实实验替换：评测结果显示，当前最强智能体在 Primitive 理解上接近人类低年级研究者，但在失败诊断与证据绑定环节仍存在显著差距。）

2. 相关工作

2.1 面向密码学的大语言模型评测

现有面向密码学的大语言模型评测主要关注知识理解、经典密码解密、密码题目求解或一般密码学推理。这类工作的重要价值在于，它们首次将密码学作为大语言模型能力边界的评估领域，并揭示了模型在符号推理、长程依赖和精确计算方面的不足。然而，这些评测通常以问题—答案为基本单元，缺少对研究过程中间工件的要求。例如，模型可能被要求给出某个密文的明文，或回答某个密码学概念，但通常不要求其提交可执行模型、求解器日志、独立验证报告或 claim-evidence 矩阵。因此，这些 benchmark 更适合衡量“密码学知识或解题能力”，而不足以衡量“密码分析研究能力”。

2.2 网络安全与软件工程智能体评测

SWE-bench、NYU CTF Bench、Cybench 等 benchmark 将评测对象从单次回答扩展到真实环境中的行动。SWE-bench 要求系统在真实 GitHub 仓库中生成补丁，并通过测试判断修复是否成功；CTF 与网络安全 benchmark 要求智能体在沙箱环境中执行命令、观察反馈并获取 flag。这类 benchmark 的方法学启发在于：评测应依赖真实任务、可执行环境、隐藏测试和统一 harness。然而，对称密码分析与软件修复或 CTF 攻击并不等同。密码分析研究的输出往往不是一个二值 flag，而是 reduced-round trail、distinguisher、bound、key-recovery 草案、复杂度估计和科学结论。这些结果必须被放在攻击模型、轮数范围、求解器状态和证据边界中解释。因此，CryptoBench-SC 借鉴可执行验证思想，但必须设计更适合密码分析研究工件的验证体系。

2.3 自动化对称密码分析工具

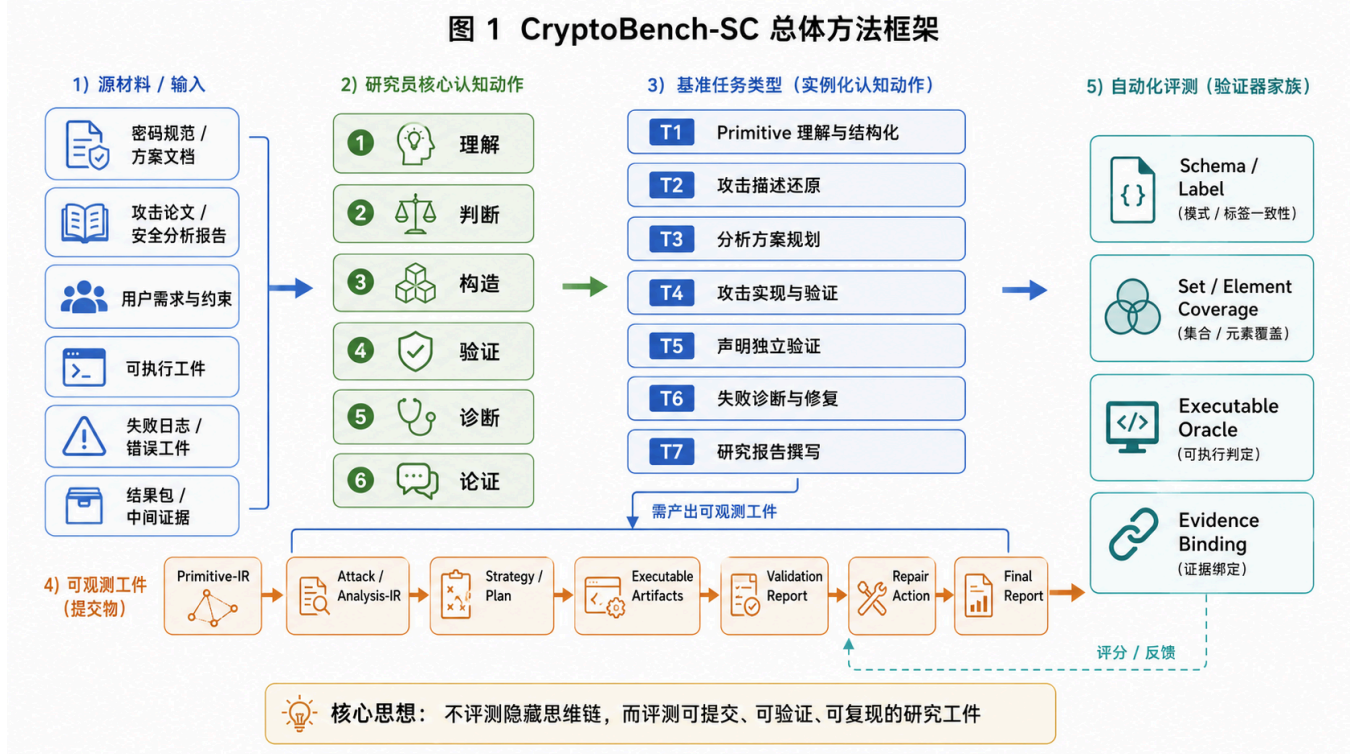
自动化密码分析工具为 CryptoBench-SC 的可执行验证提供了领域基础。现有工具链通常将对称密码原语表示为组件图或中间表示，再生成 SAT、SMT、CP 或 MILP 模型以搜索差分或线性 trail，或评估安全边界。此类工具表明，对称密码分析可以被部分形式化为“原语表示—分析任务—约束建模—求解—验证—报告”的工件链条。然而，这些工具本身并不是智能体评测框架。它们解决的是“如何执行某类分析”，而不是“如何评估一个智能体是否知道何时、为何以及

如何使用这些分析”。CryptoBench-SC 将这些工具视为后端生态的一部分，而不是将 benchmark 绑定到某一个后端。

3. CryptoBench-SC 方法

本节给出 CryptoBench-SC 的核心方法。与报告式地枚举题目类型不同，我们将 benchmark 形式化为一个由任务、产物、标签、验证器和评分函数组成的测量系统。其关键设计目标是：在不强制智能体暴露隐藏思维链的前提下，评估其研究过程是否正确、可信和可复现。

图 1 CryptoBench-SC 总体方法框架



3.1 评测概念：从隐藏思维到可观察研究产物

CryptoBench-SC 评测的不是智能体的隐藏 chain-of-thought，而是其在解决对称密码分析任务时产生的可观察研究产物。这个设计选择来自两个事实。首先，不同智能体的内部推理形式差异很大，有些系统显式规划，有些系统依赖工具反馈，有些系统使用反思循环，有些系统直接生成代码并执行。若 benchmark 要求固定推理轨迹，则会不公平地偏向某类 agent scaffold。其次，自然语言推理文本本身不一定可信，模型可能在事后生成看似合理的解释。因此，CryptoBench-SC 不要求评测不可见思维，而要求智能体提交能够被验证的外显工件。

形式化地，给定任务 t ，智能体接收可见输入 x_t ，并提交一组产物

$$Y_t = \{y_{t,1}, y_{t,2}, \dots, y_{t,m}\}.$$

这些产物可以是 Primitive-IR、Attack-IR、strategy.json、MILP/SAT/CP/OCP 工件、执行日志、验证报告、复杂度分析或最终报告。每个产物对应一个或多个隐藏标签集合 H_t 和验证器集合 V_t 。评测函数并不访问智能体内部状态，而只根据

$$S_t = \mathcal{E}(Y_t; H_t, V_t)$$

计算得分。由此，CryptoBench-SC 将“中间思考是否正确”转化为“中间产物是否能通过独立验证”。

3.2 认知动作驱动的题目划分

许多领域 benchmark 以内容类别作为题目划分依据。对称密码分析中自然可以按差分分析、线性分析、积分分析、boomerang 分析或代数攻击划分。然而，内容类别只能说明“题目涉及什么密码学内容”，不能说明“题目在评测智能体的哪种研究能力”。本文采用另一种划分依据：人类密码分析研究员完成任务时反复执行的核心认知动作。

我们将这些动作归纳为理解、判断、构造、验证、诊断和论证。理解是将外部材料转化为结构化事实；判断是在多种分析路径中选择合适策略；构造是将抽象方案落实为可执行工件；验证是独立检查结果或声明；诊断是在失败时定位根因并修复；论证是将证据组织成可复核的科学结论。该划分并不声称所有研究者都按照固定顺序思考，而是认为这些动作对应了对称密码分析研究中反复出现的可观察产物。

基于上述动作，CryptoBench-SC 定义七类题型。T1 Primitive 理解与结构化要求智能体从规范中恢复 Primitive-IR；T2 攻击描述还原要求从论文中恢复 Attack-IR；T3 分析方案规划要求在目标和资源约束下选择策略；T4 攻击实现与验证要求生成并运行可执行工件；T5 声明独立验证要求审计外部 claim；T6 失败诊断与修复要求定位并修复错误工件；T7 研究报告撰写要求将中间证据组织为审慎结论。表 1 给出题型与认知动作之间的对应关系。

表 1：题型与认知动作对应关系。

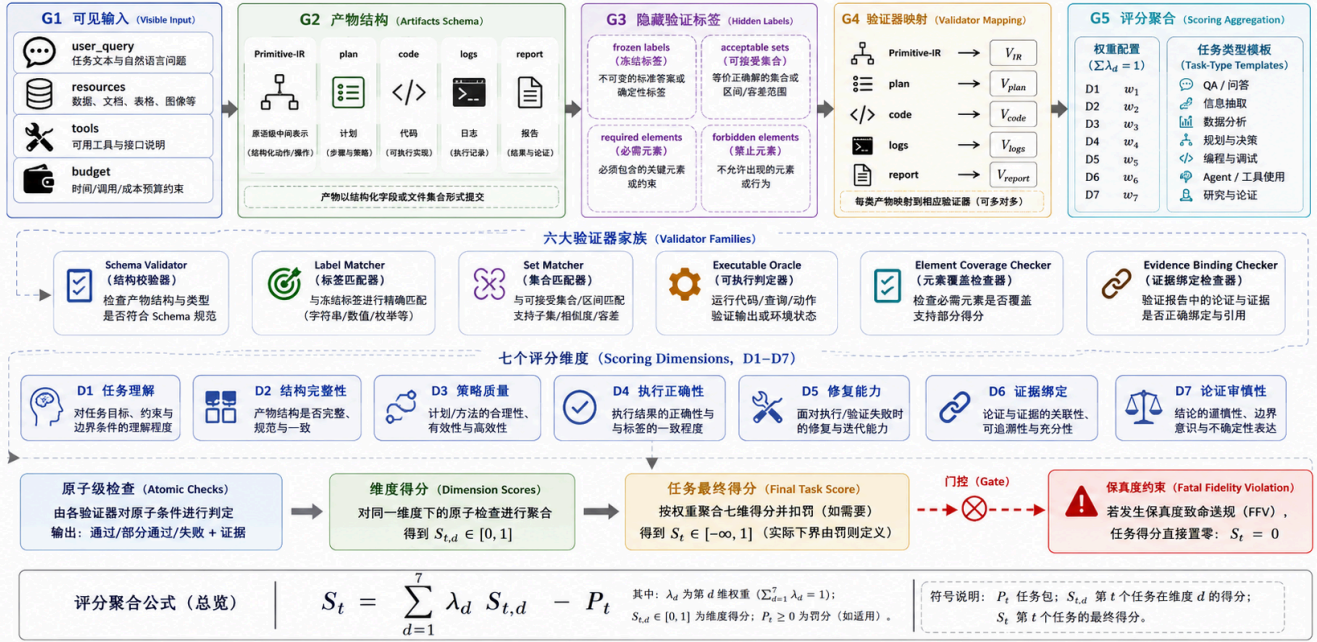
题型	名称	主导认知动作	典型产物
T1	Primitive 理解与结构化	理解	Primitive-IR
T2	攻击描述还原	理解	Attack-IR、claim boundary
T3	分析方案规划	判断	analysis plan、strategy
T4	攻击实现与验证	构造、验证	executable artifacts、logs、validation
T5	声明独立验证	验证	verification protocol、audit report
T6	失败诊断与修复	诊断	diagnosis、repair action、re-validation
T7	研究报告撰写	论证	final report、claim-evidence matrix

这样的题型设计使 CryptoBench-SC 可以同时进行原子评测与复合评测。原子题用于定位某个认知动作上的能力短板；复合题由多个题型串联而成，用于近似完整研究工作流。重要的是，题型并不与密码分析内容——绑定。同一题型可以承载差分、线性、积分或 boomerang 内容；反过来，同一个差分分析材料也可以派生出 T1、T2、T4、T5、T6 和 T7 等不同题型。

3.3 任务包表示：从题目到可验证对象

图 2 Task Package 与评分协议

$$P_t = (G_1, G_2, G_3, G_4, G_5)$$



每道题被组织为一个 task package

$$P_t = (G_1, G_2, G_3, G_4, G_5),$$

其中五个字段组分别定义输入条件、产物结构、隐藏验证标签、验证器映射和评分聚合规则。该设计的基本原则是：每个字段必须服务于某种评分机制；无法被验证或无法用于错误定位的字段不进入正式 task package。

G_1 定义智能体可见的评测输入，包括自然语言任务描述、提供资源、可用工具、计算预算和期望产出形式。该组字段不直接参与评分，但决定评测的可比性。例如，若某个 agent 获得完整论文而另一个只获得摘要，则两者得分不可直接比较。

G_2 定义智能体必须提交的产物结构。CryptoBench-SC 采用“形式与内容分离”的策略：形式由冻结的 JSON Schema 或文件布局约束，内容由语义验证器检查。这样可以将格式错误与内容错误区分开来。例如，一个 Primitive-IR 可能 schema 合规，但 S-box 表错误；一个报告可能结构完整，但存在 unsupported claim。

G_3 存储评测期不可见的隐藏验证信息。它包含四个子层：唯一答案字段使用 frozen labels；多解字段使用 acceptable sets；开放文本使用 required elements；风险输出使用 forbidden elements。该设计将开放性研究产物转化为可验证对象。例如，在 T3 分析方案规划中，MILP、CP 或 SAT iterative search 可能都属于合理后端，因此使用 acceptable set，而不是单一标准答案。在 T7 报告撰写中，“不得将 feasible 写成 optimal”“不得将 reduced-round trail 写成 full-cipher break”则作为 forbidden elements。

G_4 指定产物字段与验证器之间的映射。为保证可追溯性，每个被评分字段必须映射到确定的验证器输出。最终得分中的每一项都可以反向追溯到某个产物字段、某个验证器和某个原子判定。

G_5 定义评分聚合规则。CryptoBench-SC 的一个关键选择是：权重由题型模板决定，而不是由单题任意设定。同一题型共享一套维度权重，从而保证横向可比性。个别任务可以激活或禁用某些检查点，但不能随意改变题型级评分语义。

3.4 中间节点提取：证据链而非固定流水线

CryptoBench-SC 不要求所有智能体按照同一条固定流程执行任务。相反，每道题定义一组必须提交的可观察节点。节点来自完成该题所需的证据链，而不是来自对人类思维链的猜测。

对于一项差分 trail 搜索任务，人类专家判断其可信通常需要看到以下证据：分析对象是否正确、攻击目标是否正确、策略是否适配、后端模型是否保真、求解器是否真实执行、trail 是否通过 DDT 与线性层 replay、权重是否可重新计算、报告是否准确解释 solver status。CryptoBench-SC 将这些证据转化为节点，例如 Analysis-IR、Strategy Selection、Backend Artifact、Execution Result、Validation Report 和 Claim Boundary。智能体可以用任意顺序生成它们，但最终必须提交这些节点对应的产物。

节点分为三类。必需节点是评分所必需的产物，例如 T4 中的执行结果和验证报告。可选节点用于额外诊断或成本分析，例如 scalability assessment。隐藏验证节点不一定要求智能体显式提交，但由评测器内部计算，例如 DDT consistency、P-layer replay 和 weight recomputation。通过这种设计，CryptoBench-SC 保持了 agent 解题路径的灵活性，同时保证评测证据链的完整性。

3.5 中间表示：Primitive-IR 与 Attack/Analysis-IR

CryptoBench-SC 引入中间表示的目的不是为了形式主义，而是为了将研究语义与具体后端解耦。Primitive-IR 表示“分析对象是什么”，Attack/Analysis-IR 表示“要对它做什么分析”。前者包含 block size、state representation、round function、S-box、linear layer、key addition 和 round configuration；后者包含 attack family、goal type、rounds、scenario、input/output difference、objective、success criterion、allowed backends、validation requirements 和 claim boundary。

中间表示的正确性由三类检查共同决定。第一类是字段级正确性，例如 S-box 表、轮数或 block size 是否匹配。第二类是结构一致性，例如 P-layer 是否为双射、状态划分与 block size 是否一致、single-key 模型下 round-key difference 是否为零。第三类是任务语义一致性，例如后续后端模型是否忠实实现 Analysis-IR，报告中的 claim 是否仍然遵守 Analysis-IR 的 claim boundary。

对 IR 的评测不仅用于给分，还用于错误定位。若 agent 在 T4 执行失败，benchmark 可以根据 IR 是否正确区分“任务理解错误”和“后端实现错误”。若 IR 正确但模型错误，问题在 construction；若模型正确但报告 overclaim，问题在 argumentation。这样，CryptoBench-SC 能够提供比最终正确率更细粒度的诊断信号。

3.6 策略选择与策略效率

对称密码分析任务通常存在多条可行路线。以差分 trail 搜索为例，agent 可以使用 MILP、SAT、SMT、CP、OCP、CLAASP、自写分支限界或暴力枚举。不同路线在正确性、可扩展性和可证明性上差异明显。CryptoBench-SC 因此不固定后端，而是要求 agent 显式提交 strategy artifact，并评估其策略选择是否合理。

对任务 t ，构建阶段预先给出四类策略集合：

$$S_t^{pref}, S_t^{acc}, S_t^{disc}, S_t^{forb},$$

分别表示优选策略、可接受策略、不推荐但保真的策略和禁止策略。优选策略通常对应专家认为最适合任务目标与规模的路线；可接受策略表示能够解决任务但可能效率或证明能力较弱；不推荐策略可能在小规模实例上保真但不可扩展；禁止策略包括降轮、改变 key model、忽略核心组件或把 exact trail 改成 truncated proxy 等不保真 shortcut。

策略分由任务适配性、专家策略一致性、复杂度意识、实际成本和保真度共同决定：

$$S_{\text{strat}} = 0.25S_{\text{compat}} + 0.20S_{\text{expert}} + 0.20S_{\text{complexity}} + 0.15S_{\text{cost}} + 0.20S_{\text{fidelity}}.$$

其中 S_{cost} 不是单纯 wall-clock time，而是归一化成本：

$$S_{\text{cost}} = \exp \left(- \sum_i \lambda_i \log(1 + c_i/b_{t,i}) \right),$$

c_i 表示 agent 的时间、solver time、token、工具调用或内存成本， $b_{t,i}$ 为该任务参考成本。该设计避免将“快但改题”的路线误判为优秀策略。

3.7 验证器与可执行 oracle

CryptoBench-SC 使用确定性验证器作为评分核心，而不依赖常规 LLM-as-judge。验证器分为六类：Schema Validator 检查产物结构；Label Matcher 检查唯一答案字段；Set Matcher 检查多解字段是否落入可接受集合；Executable Oracle 在沙箱中执行代码或模型并比对结果；Element Coverage Checker 检查开放文本是否覆盖必要要素或包含禁止要素；Evidence Binding Checker 检查报告中的每条 claim 是否链接到具体证据。

在可执行任务中，Executable Oracle 是关键。对于差分 trail 任务，oracle 通常包含 DDT 检查、P-layer replay、权重重计算和 solver status 检查。若 agent 声称某 trail 是 optimal，oracle 必须检查 solver 是否给出 optimal certificate；若只得到 feasible solution，则报告中不得出现 optimal claim。对于 key-recovery 任务，oracle 检查猜测子密钥范围、候选集合、复杂度估计和是否优于穷举。对于失败修复任务，oracle 不仅检查修复后是否运行成功，还检查修复是否保持原任务语义。

3.8 评分函数与保真度约束

CryptoBench-SC 的评分采用组合式结构。每个维度由若干原子检查点组成，每个检查点输出

$$z_{t,j} \in \{0, 1\}.$$

维度分数为

$$S_{t,d} = \sum_{j \in J_{t,d}} \alpha_{t,j} z_{t,j},$$

其中 $J_{t,d}$ 表示任务 t 在维度 d 上的检查点集合。最终任务分数为

$$S_t = \sum_{d=1}^7 \lambda_{\tau(t),d} S_{t,d} - P_t,$$

其中 $\tau(t)$ 为题型， $\lambda_{\tau(t),d}$ 为题型级权重， P_t 为保真度惩罚。

保真度惩罚是 CryptoBench-SC 的核心机制之一。某些错误不能仅作为普通扣分处理，因为它们改变了任务本质。例如降低轮数、改变 single-key/related-key 模型、删除线性层、把 exact trail 换成 truncated bound、把 reduced-round 结果写成 full-cipher break 等行为会触发强惩罚。对于致命不保真行为，任务得分被门控为零：

$$S_t = 0 \quad \text{if FatalFidelityViolation}(Y_t) = 1.$$

对于较轻的 claim overreach，则从 D6 或 D7 中扣除相应分数。通过保真度约束，CryptoBench-SC 鼓励 agent 自主选择方法，但不允许通过改题获得成功。

3.9 难度建模与校准

任务难度不由策展者主观指定，而由可分解难度维度与试点校准共同确定。对任务 t ，我们标注源理解、抽象形式化、实现、验证和报告五个维度的难度：

$$d_t^{(u)}, d_t^{(a)}, d_t^{(r)}, d_t^{(v)}, d_t^{(g)} \in \{0, 1, 2\}.$$

综合难度为

$$D_t = w_u d_t^{(u)} + w_a d_t^{(a)} + w_r d_t^{(r)} + w_v d_t^{(v)} + w_g d_t^{(g)}.$$

初始权重来自 workflow 先验：实现维度在首版中权重略高，因为后端建模、求解器配置和可执行验证是最易失败的环节；报告维度略低，因为并非所有任务都要求完整报告。（占位假设，待真实实验替换：默认权重设为 (0.20, 0.20, 0.25, 0.20, 0.15)。）

正式发布前，权重与 Easy/Medium/Hard 阈值通过专家首次求解时间和 pilot 失败率校准。设 FST_t 为专家首次求解时间， $FailRate_t$ 为 pilot 中失败比例，我们构造经验难度目标：

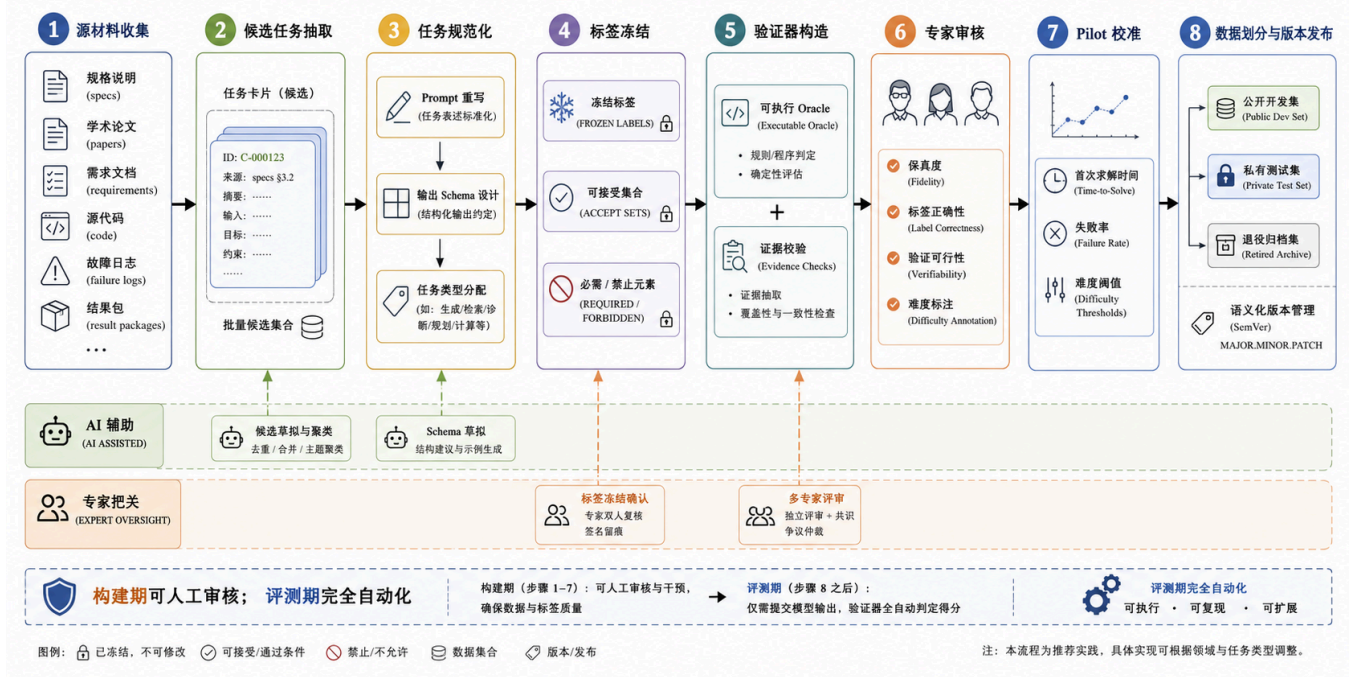
$$y_t = \alpha z(\log FST_t) + (1 - \alpha) z(FailRate_t).$$

随后在单纯形约束下拟合权重，使 $D_t(w)$ 与 y_t 的偏差最小。正式版本还报告权重敏感性分析，检查小幅扰动下任务难度标签和系统排名是否稳定。

4. Benchmark 构建流程

CryptoBench-SC 的构建目标不是一次性生成题目，而是建立从真实材料到冻结评测任务的可审计流水线。构建流程包括源材料收集、候选任务抽取、任务规范化、标签与验证器构造、专家审核、pilot 校准和版本发布。

图 3 Benchmark 构建与发布流水线



4.1 源材料收集

源材料来自五类真实或准真实密码分析材料：密码规范与标准文档、攻击论文与安全分析报告、分析目标与资源约束、可执行工件与失败日志、结果包与中间证据。首版以分组密码差分与线性分析为核心，同时保留 impossible differential、integral/division property、boomerang/related-key 和 key-recovery extension 的少量样本，以测试题型框架的扩展性。（占位假设，待真实实验替换：v1.0 中 differential 占 45%，linear 占 20%，impossible differential 占 10%，integral/division property 占 10%，boomerang/related-key 占 10%，key-recovery extension 占 5%。）

4.2 候选任务抽取

每份源材料首先被转化为 candidate task card。task card 记录材料来源、可评测认知动作、题型、内容标签、预期产物、可用验证器、潜在保真风险和初始难度。抽取过程可以由大模型辅助，但不得直接将大模型输出作为冻结标签。构建者使用大模型的主要方式是生成候选题面、列举可能的 required elements、提示潜在 forbidden elements，以及辅助生成初版 schema；最终标签和验证器均由人类专家审核。

同一源材料可以产生多道任务。例如一篇差分分析论文可以生成 T2 攻击描述还原题、T4 可执行复现题、T5 声明独立验证题、T7 报告撰写题；若提供错误复现脚本，还可以生成 T6 失败修复题。这样的派生方式保证内容来源真实，同时避免把整篇论文强行压缩成单一答案。

4.3 任务规范化与标签冻结

候选任务经过规范化后进入 task package。规范化包括三步。第一，题面重写为明确、可执行、边界清晰的 prompt，但不得改变原始研究问题。第二，确定产物 schema 与必需节点，确保 agent 的输出可被自动解析。第三，冻结 hidden labels、acceptable sets、required elements 和 forbidden elements。

这里最重要的原则是“可以修表达，不可以偷偷改题”。为了适配 benchmark，构建者可以精简论文背景、提供必要上下文、固定计算预算或将任务限制在 reduced-round setting；但不能将高轮数任务降成玩具题却仍声称评测原任务，不能将 related-key 改成 single-key，不能用 truncated proxy 替代 exact trail 后仍按原目标评分。若发生目标改变，必须显式创建新任务并修改标签。

4.4 验证器构造

每道正式题必须至少有一个核心验证器。对于 T1 与 T2，验证器主要检查结构字段与一致性；对于 T3，验证器检查方法选择是否落入 acceptable set、是否覆盖验证方案与风险说明；对于 T4，验证器运行代码或模型并验证数值；对于 T5，验证器执行独立计算或比对；对于 T6，验证器检查根因、修复类别与 re-validation；对于 T7，验证器解析 claim-evidence 结构并检测 overclaim。

验证器本身也需要测试。每个验证器在发布前使用正例、负例和边界例进行单元测试。正例来自参考答案或专家工件，负例包括常见错误，如漏掉 P-layer、错误 S-box、错误 key model、将 feasible 写成 optimal、将 reduced-round 结果写成 full attack。只有能稳定区分这些错误的验证器才能进入正式版本。

4.5 专家审核与一致性

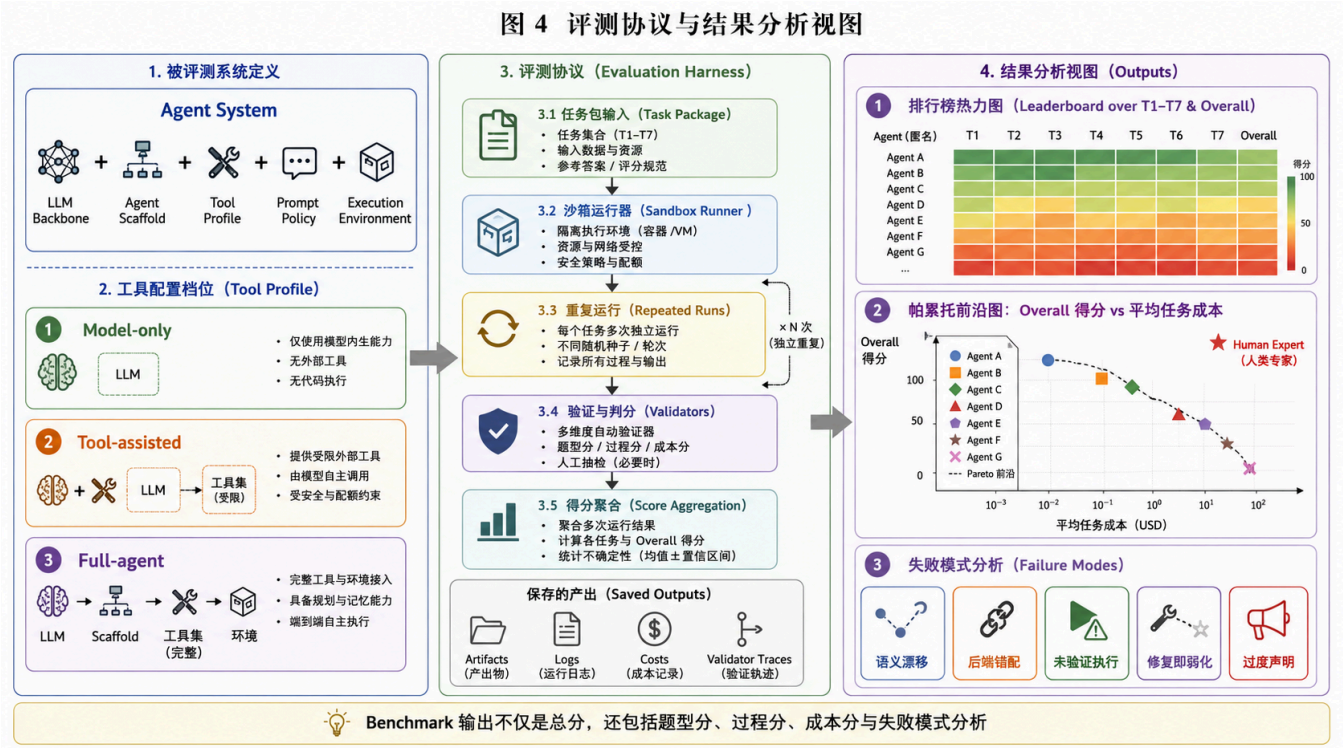
CryptoBench-SC 将主观判断前置到构建期。所有需要领域判断的标签由至少两名专家独立审核，包括 acceptable sets、required elements、forbidden elements、保真边界和难度标注。专家分歧通过讨论或第三方仲裁解决。正式发布时报告专家间一致性。（占位假设，待真实实验替换：正式数据集中 Cohen's κ 均值为 0.78，低于 0.70 的题目被重写或删除。）

专家审核并不意味着评测期依赖人工。相反，它的作用是确保评测期可以完全自动化。一旦标签冻结，所有智能体提交均由验证器评分，评测者不再临场判断答案是否合理。

4.6 数据划分、版本化与污染治理

CryptoBench-SC 采用 public development、private active test 和 retired archive 三层划分。开发集公开题面、schema 和部分参考答案，供系统开发与调试；私有测试集用于 leaderboard；退役集用于版本复现、污染分析和方法研究。版本号采用语义化管理，例如 v1.0、v1.1、v2.0。同一版本内不修改 hidden labels 与评分权重；任何修正都通过新版本发布。为降低污染，私有测试集只通过受控评测服务运行，定期退役并替换。

5. 评测协议



5.1 被测对象

CryptoBench-SC 评测的是完整 agent system，而不是孤立语言模型。一个系统被定义为

Agent = LLM backbone + agent scaffold + tool profile + prompt policy + execution environment.

该定义使我们能够同时报告现实使用视角下的系统榜单，以及控制变量视角下的模型、框架和工具消融。

5.2 工具配置

为避免将模型能力与工具能力混淆，CryptoBench-SC 定义三种工具配置。Model-only 设置不允许执行外部密码分析工具，主要测试理解、形式化、规划与报告。Tool-assisted 设置允许使用 Python、求解器和指定密码分析工具，测试可执行实现与验证。Full-agent 设置允许完整工具编排、检索、执行、修复循环和日志分析，用于模拟最接近真实研究助理的工作方式。所有配置均有固定预算，包括最大时间、最大工具调用、最大 token 和最大内存。

5.3 运行流程

每个 agent-task pair 在沙箱中独立运行。runner 向智能体提供可见 task package，收集提交工件，并依次执行 schema 检查、标签匹配、集合匹配、沙箱执行、oracle 验证、证据绑定检查和评分聚合。所有中间日志、工具调用、validator 输出和成本指标均被保存。由于智能体具有随机性，每个 pair 至少运行多次，并报告均值、方差和 pass@k。（占位假设，待真实实验替换：正式实验中每个 pair 运行 3 次。）

5.4 人类基线

人类基线用于判断任务是否确实位于研究构念范围内。参与者为具备对称密码或密码分析背景的研究人员，在相同输入与时间预算下完成任务。人类基线不是作为绝对上界，而是作为解释 agent 得分的参照。若某类任务专家也普遍失败，则该任务可能超出当前 benchmark 的合理难度范围，需要降级为 challenge 或从主榜移除。

5.5 统计分析

除总分外，CryptoBench-SC 报告题型分、认知动作分、内容标签分、工具配置分和成本分。为评估 benchmark 区分度，我们计算任务通过率、系统排名稳定性、题目鉴别度、过程分与最终分相关性、以及权重扰动下的 Spearman 排名相关。必要时可使用 IRT 模型估计题目难度与鉴别参数：

$$P(z_{i,t} = 1) = \sigma(a_t(\theta_i - b_t)),$$

其中 θ_i 为系统能力， b_t 为任务难度， a_t 为任务鉴别度。高鉴别度任务对于 leaderboard 更有价值，低鉴别度或专家分歧大的任务需要重新审核。

6. 实验

本节数值均为占位写作示例，正式论文必须以真实评测替换。

6.1 实验设置

我们使用 CryptoBench-SC v1.0 评测 12 个系统，覆盖闭源前沿模型、开源模型、通用 agent scaffold、代码型 agent scaffold 和密码工具增强 agent。（占位假设，待真实实验替换：评测系统包括 GPT-5 系列 agent、Claude Opus 系列 agent、Gemini 系列 agent、DeepSeek 系列 agent、Qwen 开源 agent，以及三组消融系统。）人类基线由 4 名密码学研究人员完成，其中包括博士后、青年教师和高年级博士。（占位假设，待真实实验替换：人类基线任务子集为 60 道，覆盖 7 个题型。）

6.2 主结果

表 2 给出 overall score 与各题型得分。（占位假设，待真实实验替换：以下为虚拟示例。）

系统	Overall	T1	T2	T3	T4	T5	T6	T7
Human Expert	82.7	91.4	85.2	86.1	79.6	84.3	75.8	86.7
Agent-A Full	41.8	66.5	49.1	43.7	38.2	35.6	24.3	34.9
Agent-B Full	40.9	62.7	46.8	39.5	41.1	34.2	28.7	33.1
Agent-C Tool-assisted	33.1	55.2	39.4	32.8	33.6	27.1	19.4	24.8
Best Model-only	24.6	51.7	32.6	29.1	10.3	18.5	7.9	22.4

该结果呈现三个趋势。第一，工具访问显著提升 T4，但不能自动提升 T6 和 T7，说明执行能力与诊断、论证能力并非同一能力。第二，智能体在 T1 上得分较高，表明其能够从规范和文本中抽取基本结构，但在 T5 和 T6 中明显下降，说明独立验证与失败修复仍是瓶颈。第三，人类专家与智能体之间的差距在 T6 和 T7 最大，表明当前系统最缺乏的是保真修复和证据边界控制。

6.3 最终答案与过程分的错位

CryptoBench-SC 的一个核心发现是，最终答案正确并不意味着研究过程正确。（占位假设，待真实实验替换：在所有 final-answer 通过的 agent 提交中，18.4% 未通过至少一个关键中间验证器，最常见错误包括未验证 P-layer replay、将 TIME_LIMIT 结果表述为 optimal、以及未说明 reduced-round 限制。）如果只使用 final-only 评分，这些错误会被掩盖。分层评分则能够区分真正可复核的成功与偶然或不保真的成功。

6.4 工具与策略消融

我们进一步比较不同工具配置。（占位假设，待真实实验替换：相较 Model-only，Tool-assisted 在 T4 上提升 17.8 分，但在 T7 上仅提升 2.4 分；加入证据绑定模块后 T7 提升 11.6 分；加入 repair loop 后 T6 提升 13.2 分。）这说明密码分析智能体的改进需要针对不同认知动作配置不同模块：求解器提升构造与执行，证据追溯提升论证，错误日志分析与保真约束提升诊断。

6.5 失败模式分析

我们观察到五类主要失败。第一是语义漂移，即 agent 为了跑通而降低轮数或改变 key model。第二是后端错配，即对优化问题使用不合适的可行性搜索或对 ARX 结构套用不合适的 S-box 模板。第三是未验证执行，即直接相信 solver 输出而不做 DDT 或 P-layer replay。第四是修复即弱化，即通过删除约束或简化任务修复失败。第五是过度声明，即将 trail 写成 differential，将 distinguisher 写成 key recovery，或将 reduced-round 结果写成 full break。上述失败模式正是 final-answer benchmark 难以揭示的过程性错误。

6.6 区分度与稳定性

为了检验 benchmark 是否能稳定区分系统，我们报告系统排名在重复运行和权重扰动下的稳定性。（占位假设，待真实实验替换：三次重复运行的 overall score 平均标准差为 2.1；权重在 ± 0.05 范围内扰动时，系统排名 Spearman 相关系数为 0.96；IRT 分析显示 71% 的任务具有中高鉴别度。）这些结果支持 CryptoBench-SC 作为比较不同 agent 系统的稳定评测工具。

7. 讨论

CryptoBench-SC 的核心价值在于，它将“智能体是否会做密码分析”这一笼统问题拆解为一组可测能力。一个系统可能擅长读规范，但不会选策略；可能会调用求解器，但不会验证；可能会生成报告，但不能绑定证据。通过认知动作和题型分解，我们能够将总分背后的失败原因显式化。

该 benchmark 也对智能体设计给出直接启示。若系统在 T4 低分，应优先改进工具调用和模型生成；若在 T6 低分，应加入错误日志理解、保真修复约束和 re-validation loop；若在 T7 低分，应加入 claim-evidence 结构和过度声明检测；若策略分低，则需要增强任务—后端匹配和复杂度意识。换言之，CryptoBench-SC 不只是排行榜，也可以作为 agent 框架改进的诊断工具。

同时，CryptoBench-SC 仍有局限。首版覆盖范围不可能穷尽全部对称密码分析方向；确定性验证器难以覆盖所有开放性科学论证；可执行 oracle 的构建成本较高，可能使早期版本偏向可形式化任务；人类基线昂贵且受参与者背景影响；benchmark 衡量的是有边界的研究任务，而非长期原创密码分析项目。

8. 结论

本文提出 CryptoBench-SC，一个面向对称密码分析智能体研究能力的认知动作驱动 benchmark。我们将对称密码分析研究员的核心认知动作转化为七类可构造、可提交、可验证、可评分的任务类型，并通过结构化 task package、隐藏标签、确定性验证器、可执行 oracle、证据绑定检查和保真度约束，实现对智能体研究工作流能力的细粒度评测。

（占位假设，待真实实验替换：pilot 结果显示，当前最强智能体与人类研究员仍存在显著差距，尤其体现在失败修复和证据约束下的报告撰写方面。）这一结果表明，下一代密码分析智能体的发展不应只追求更强的单轮推理或更大的模型规模，还应强化保真修复、独立验证、策略选择和证据绑定能力。CryptoBench-SC 为衡量这些能力提供了可复现、可审计的评测基础。

附录 A: 示例 Task Package

```
{
  "task_id": "CBSC-T4-MINISPN-DIFF-003",
  "task_type": "T4_Attack_Realization_and_Verification",
  "content_tags": {
    "primitive_family": "SPN",
    "attack_family": "differential",
    "goal_type": "minimum_weight_trail",
    "backend_options": ["MILP", "CP", "SAT"]
  },
  "G1_task_inputs": {
    "user_query": "Find and verify a 3-round minimum-weight differential trail for MiniSPN-16 with input difference 0x0001.",
    "provided_resources": ["primitive_ir.json", "analysis_ir.json"],
    "available_tools": ["python", "milp_solver", "cp_sat"],
    "compute_budget": "10 minutes"
  },
  "G2_artifact_structure": {
    "required": ["strategy.json", "model.py", "solver_log.txt", "result.json", "validation_report.json"]
  },
  "G3_hidden_validation_labels": {
    "frozen_labels": {
      "rounds": 3,
      "input_difference": "0x0001",
      "key_model": "single-key"
    },
    "acceptable_set": {
      "backend": ["MILP", "CP", "SAT_iterative_weight_bound"]
    },
    "required_elements": [
      "DDT consistency check",
      "P-layer replay check",
      "weight recomputation",
      "solver status interpretation"
    ],
    "forbidden_elements": [
      "reduce_round_count",
      "omit_P_layer",
      "report_feasible_as_optimal"
    ]
  },
  "G4_validator_mapping": {
    "strategy.json": "SetMatcher",
    "model.py": "ExecutableOracle",
    "result.json": "LabelMatcher+NumericalCheck",
    "validation_report.json": "ElementCoverageChecker"
  },
  "G5_score_aggregation": {
    "D3_planning_quality": 0.20,
    "D4_execution_correctness": 0.45,
  }
}
```

```
"D6_evidence_binding": 0.20,  
  "D7_claim_prudence": 0.15  
}  
}
```

参考文献

- [1] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR, 2024.
- [2] Minghao Shao et al. *NYU CTF Bench: A Scalable Open-Source Benchmark Dataset for Evaluating LLMs in Offensive Security*. NeurIPS Datasets and Benchmarks, 2024.
- [3] Dan Boneh et al. *Cybench: A Framework for Evaluating Cybersecurity Capabilities and Risks of Language Models*. arXiv preprint, 2024/2025.
- [4] Yu Li et al. *CipherBank: Exploring the Boundary of LLM Reasoning Capabilities through Cryptography Challenges*. arXiv preprint, 2025.
- [5] Yu Wang et al. *AlCrypto: A Comprehensive Benchmark for Evaluating Cryptography Capabilities of Large Language Models*. arXiv preprint, 2025.
- [6] Emanuele Bellini et al. *CLAASP: A Cryptographic Library for the Automated Analysis of Symmetric Primitives*. Selected Areas in Cryptography, 2023.
- [7] James Nechvatal et al. *Report on the Development of the Advanced Encryption Standard (AES)*. NIST, 2001.
- [8] NIST. *Lightweight Cryptography Standardization Process: Submission Requirements and Evaluation Criteria*. 2018.
- [9] Eli Biham and Adi Shamir. *Differential Cryptanalysis of DES-like Cryptosystems*. Journal of Cryptology, 1991.
- [10] Mitsuru Matsui. *Linear Cryptanalysis Method for DES Cipher*. EUROCRYPT, 1993.